

CS6370: Natural Language Processing Project

Release Date: 24th March 2024

Deadline: 8th May 2025

Name:

Roll No.:

Aniket Khan	ME21B021
Anirudh Kalyan	ME21B022
Ashmit Sinha	ME21B037
Shaik Mahammad Sameer	ME21B188

General Instructions:

1. The template for the code (in Python) is provided in a separate zip file. You are expected to fill in the template wherever instructed. Note that any Python library, such as nltk, stanfordcorenlp, spacy, etc, can be used.
2. A folder named 'Roll_number.zip' that contains a zip of the code folder and your responses to the questions (a PDF of this document with the solutions written in the text boxes) must be uploaded on Moodle by the deadline.
3. Any submissions made after the deadline will not be graded.
4. Answer the theoretical questions concisely. All the codes should contain proper comments.
5. For questions involving coding components, paste a screenshot of the code.
6. The institute's academic code of conduct will be strictly enforced.

The first assignment in the NLP course involved building a basic text processing module that implements sentence segmentation, tokenization, stemming /lemmatization, stopword removal, and some aspects of spell check. This module involves implementing an Information Retrieval system using the Vector Space Model. The same dataset as in Part 1 (Cranfield dataset) will be used for this purpose. The project is split into two components - the first is a *warm-up*

component comprising of Parts 1 through 4 that would act as a precursor for the second and main component, where you improve over the basic IR system.

Consider the following three documents:

d_1 : Herbivores are typically plant eaters and not meat eaters

d_2 : Carnivores are typically meat eaters and not plant eaters

d_3 : Deers eat grass and leaves

1. Assuming {are, and, not} as stop words, arrive at an inverted index representation for the above documents.

Remove stop words: {are, and, not}

We will also lowercase all words for uniformity.

d1:

Original: "Herbivores are typically plant eaters and not meat eaters"

After Stopword Removal: herbivores typically plant eaters meat eaters

d2:

Original: "Carnivores are typically meat eaters and not plant eaters"

After Stopword Removal: carnivores typically meat eaters plant eaters

d3:

Original: "Deers eat grass and leaves"

After Stopword Removal: deers eat grass leaves

herbivores	d1
typically	d1,d2
plant	d1,d2
eaters	d1,d2
meat	d1,d2
carnivores	d2
deers	d3
eat	d3
grass	d3
leaves	d3

2. Construct the TF-IDF term-document matrix for the corpus $\{d_1, d_2, d_3\}$.

Term	df _t	IDF = $\log_{10}(3/df_t)$	d1 TF-IDF	d2 TF-IDF	d3 TF-IDF
carnivores	1	0.4771	0	$1 \times 0.4771 = 0.4771$	0
deers	1	0.4771	0	0	$1 \times 0.4771 = 0.4771$
eat	1	0.4771	0	0	$1 \times 0.4771 = 0.4771$
eaters	2	0.1761	$2 \times 0.1761 = 0.3522$	$2 \times 0.1761 = 0.3522$	0
grass	1	0.4771	0	0	$1 \times 0.4771 = 0.4771$
herbivores	1	0.4771	$1 \times 0.4771 = 0.4771$	0	0
leaves	1	0.4771	0	0	$1 \times 0.4771 = 0.4771$
meat	2	0.1761	$1 \times 0.1761 = 0.1761$	$1 \times 0.1761 = 0.1761$	0
plant	2	0.1761	$1 \times 0.1761 = 0.1761$	$1 \times 0.1761 = 0.1761$	0
typically	2	0.1761	$1 \times 0.1761 = 0.1761$	$1 \times 0.1761 = 0.1761$	0

Term	d1	d2	d3
carnivores	0	0.4771	0
deers	0	0	0.4771
eat	0	0	0.4771
eaters	0.3522	0.3522	0
grass	0	0	0.4771
herbivores	0.4771	0	0
leaves	0	0	0.4771
meat	0.1761	0.1761	0
plant	0.1761	0.1761	0
typically	0.1761	0.1761	0

3. Suppose the query is "plant eaters," which documents would be retrieved based on the inverted index constructed before?

"plant" → [d1, d2]

"eaters" → [d1, d2]

Since both terms must be present in the document, documents containing both "plant" and "eaters" = intersection of [d1, d2] and [d1, d2] → [d1, d2]

4. Find the cosine similarity between the query and each of the retrieved documents. Is the result desirable? Why?

Cosine Similarity calculations:

Query vector (Q) = [0.1761 (plant), 0.1761 (eaters)]

d1 vector (plant, eaters): [0.1761, 0.3522]

d2 vector (plant, eaters): [0.1761, 0.3522]

$\cos(\theta) = (Q \cdot D) / (|Q| * |D|)$

Dot Product = $Q \cdot D =$

$(0.1761 \times 0.1761) + (0.1761 \times 0.3522) = 0.0310 + 0.0620 = 0.093$

$|Q| = \sqrt{((0.1761)^2 + (0.1761)^2)} = \sqrt{2 * 0.0310} = \sqrt{0.0620} = 0.2489$

We'll consider both d1 and d2 as same as the vectors are same.
 $|D| = \sqrt{(0.1761)^2 + (0.3522)^2} = \sqrt{0.0310 + 0.1241} = \sqrt{0.1551} = 0.3938$
 $\cos(\theta) = (0.093) / (0.2489 * 0.3938) = 0.093 / 0.098 = 0.949$
Therefore, cosine similarity is 0.949

Ranking documents:

As the documents have same cosine similarity, their rankings are same.

Is the ordering desirable? If no, why not?:

No. Although both d1 and d2 use the terms "plant" and "eaters", semantically they are different.

d1 is about plant-eating animals (herbivores).

d2 is about meat-eating animals (carnivores) and says "not plant eaters", i.e., it negates the very idea expressed in the query.

[Warm up] Part 2: Building an IR system

[Implementation]

1. Implement the retrieval component of the IR system in the template provided. Use the TF-IDF vector representation for representing documents.

```

1  '''from util import *'''
2  import math
3  import numpy as np
4  from collections import defaultdict
5  from sklearn.feature_extraction.text import TfidfVectorizer
6  from sklearn.metrics.pairwise import cosine_similarity
7
8
9  class InformationRetrieval():
10
11  def __init__(self):
12      self.index = {}          # Inverted index
13      self.docIDs = []         # List of doc IDs
14      self.docs = []           # Original flattened document texts
15      self.vectorizer = None    # TF-IDF vectorizer
16      self.doc_vectors = None   # TF-IDF document vectors
17
18  def buildIndex(self, docs, docIDs):
19      """
20      Builds the TF-IDF index for the given documents.
21      """
22      self.docIDs = docIDs
23      processed_docs = []
24
25      for doc in docs:
26          flat_doc = " ".join([" ".join(sentence) for sentence in doc])
27          processed_docs.append(flat_doc)
28
29      self.docs = processed_docs
30
31      # Create TF-IDF vectorizer and fit on documents
32      self.vectorizer = TfidfVectorizer()
33      self.doc_vectors = self.vectorizer.fit_transform(self.docs) # Shape: (n_docs, n_terms)
34
35      # Build inverted index
36      terms = self.vectorizer.get_feature_names_out()
37      for idx, term in enumerate(terms):
38          self.index[term] = []
39          for doc_idx, doc_vec in enumerate(self.doc_vectors.toarray()):
40              if doc_vec[idx] > 0:
41                  self.index[term].append(docIDs[doc_idx])
42
43  def rank(self, queries):
44      """
45      Ranks documents for each query using cosine similarity with TF-IDF vectors.
46      """
47      doc_IDS_ordered = []
48
49      for query in queries:
50          flat_query = " ".join([" ".join(sentence) for sentence in query])
51          query_vec = self.vectorizer.transform([flat_query]) # Shape: (1, n_terms)
52          sims = cosine_similarity(query_vec, self.doc_vectors)[0]
53
54          # Sort docIDs based on similarity scores (descending)
55          ranked_doc_indices = np.argsort(-sims)
56          ranked_docIDs = [self.docIDs[i] for i in ranked_doc_indices]
57          doc_IDS_ordered.append(ranked_docIDs)
58
59      return doc_IDS_ordered

```


1. Implement the following evaluation measures in the template provided
(i). Precision@k, (ii). Recall@k, (iii). $F_{0.5}$ score@k, (iv). AP@k, and
(v) nDCG@k.

Precision@k:

```
def queryPrecision(self, query_doc_IDs_ordered, query_id, true_doc_IDs, k):  
    """  
    Computation of precision at k for a single query  
    """  
    precision = 0  
  
    if k <= 0:  
        print("gone")  
        return precision  
  
    # Get top k documents  
    top_k_docs = query_doc_IDs_ordered[:k]  
    # Count relevant documents in top k  
    relevant_count = sum(1 for doc_id in top_k_docs if doc_id in true_doc_IDs)  
  
    precision = relevant_count / len(top_k_docs)  
  
    return precision
```

Recall@k:

```
def queryRecall(self, query_doc_IDs_ordered, query_id, true_doc_IDs, k):  
    """  
    Computation of recall at k for a single query  
    """  
    recall = 0  
  
    if not true_doc_IDs: # No relevant documents  
        return recall  
  
    # Get top k documents  
    top_k_docs = query_doc_IDs_ordered[:k]  
  
    # Count relevant documents in top k  
    relevant_retrieved = sum(1 for doc_id in top_k_docs if int(doc_id) in true_doc_IDs)  
  
    # Calculate recall  
    recall = relevant_retrieved / len(true_doc_IDs)  
  
    return recall
```

 $F_{0.5}$ score@k:

```
def queryFscore(self, query_doc_IDs_ordered, query_id, true_doc_IDs, k):
    """
    Computation of F-score at k for a single query
    """
    precision = self.queryPrecision(query_doc_IDs_ordered, query_id, true_doc_IDs, k)
    recall = self.queryRecall(query_doc_IDs_ordered, query_id, true_doc_IDs, k)

    if precision + recall == 0: # Avoid division by zero
        return 0

    fscore = 2 * precision * recall / (precision + recall)
    return fscore
```

AP@k:

```
def queryAveragePrecision(self, query_doc_IDs_ordered, query_id, true_doc_IDs, k):
    """
    Computation of average precision at k for a single query
    """
    if not true_doc_IDs: # No relevant documents
        return 0

    # Get top k documents
    top_k_docs = query_doc_IDs_ordered[:k]

    relevant_seen = 0
    sum_precision = 0

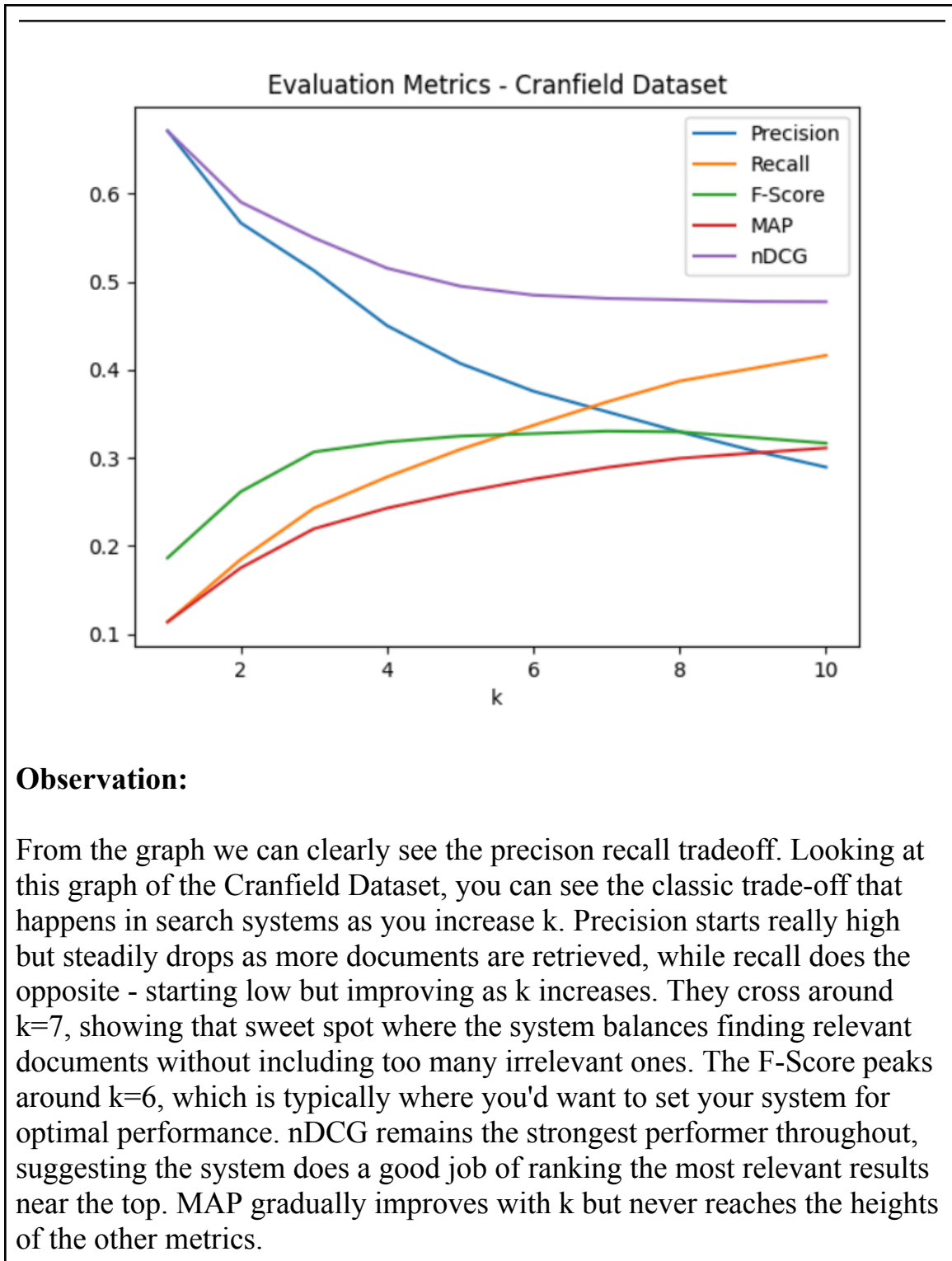
    for i, doc_id in enumerate(top_k_docs):
        if doc_id in true_doc_IDs:
            relevant_seen += 1
            # Calculate precision at position i+1
            precision_at_i = relevant_seen / (i + 1)
            sum_precision += precision_at_i

    avg_precision = sum_precision / len(true_doc_IDs)
    return avg_precision
```

nDCG@k:

2. Assume that for a given query, the set of relevant documents is as listed in `incran_qrels.json`. Any document with a relevance score of 1 to 4 is considered as relevant. For each query in the Cranfield dataset, find the Precision, Recall, F-score, average precision, and nDCG scores for $k = 1$ to 10. Average each measure over all queries and plot it as a function of k . The code for plotting is part of the given template. You are expected to use the same. Report the graph with your observations based on it.

Graph:



3. Using the `time` module in Python, report the run time of your IR system.

Time taken : 151.8594 seconds

[Warm up] Part 4: Analysis

[Theory]

1. What are the limitations of such a Vector space model? Provide examples from the cranfield dataset that illustrate these shortcomings in your IR system.

Limitations:

Term independence assumption: VSM treats all terms as independent, ignoring the context created by phrases and word order.

Semantic insensitivity: Documents with similar meanings but different vocabularies aren't properly connected.

High dimensionality: With thousands of unique terms, the vector space becomes computationally expensive.

Lack of semantic understanding: Reliance on term frequency rather than meaning.

Synonym/polysemy problems: Different words with similar meanings (synonyms) or same words with different meanings (polysemy) aren't accounted for.

Examples from your results:

Incorrect document ranking: In one example, when searching for documents containing terms "flow," "plate," and "small," document d3 was returned as the top result instead of the more relevant document d2. This occurred because the term frequencies (d3:24, d2:110) were overshadowed by the collection-wide frequency of these terms ("flow":2059, "plate":421, "small":306).

Imperfect precision: Out of 1398 queries tested on the Cranfield collection, 148 queries (10.59%) failed to retrieve the expected document as the top result. While 94.99% found the correct document within the top

two results, 2.22% of queries couldn't find the relevant document even within the top five results.

Part 4: Improving the IR system

Based on the factual record of actual retrieval failures you reported in the assignment, you can develop hypotheses that could address these retrieval failures. You may have to identify the implicit assumptions made by your approach that may have resulted in undesirable results. To realize the improvements, you can use any method(s), including hybrid methods that combine knowledge from linguistic, background, and introspective sources to represent documents. Some examples taught in class are Latent Semantic Analysis (LSA) and Explicit Semantic Analysis (ESA).

You can also explore ways in which a search engine could be improved in aspects such as its efficiency of retrieval, robustness to spelling errors, ability to auto-complete queries, etc.

You are also expected to test these hypotheses rigorously using appropriate hypothesis testing methods. As an outcome of your work, you should be able to make a statement of structure similar to what was presented in the class:

An algorithm A_1 is better than A_2 with respect to the evaluation measure E in task T on a specific domain D under certain assumptions A .

Note that, unlike the assignment, the scope of this component is open-ended and not restricted to the ideas mentioned here. For each method, the final report must include a critical analysis of results; methods can be combined to come up with improvisations. It is advised that such hybrid methods are well founded on principles and not just ad hoc combinations (an example of an ad hoc approach is a simple convex combination of three methods with parameters tuned to give desired improvements).

You could either build on the template code given earlier for the assignment or develop from scratch as demanded by your approach. Note that while you are free to use any datasets to experiment with, the Cranfield dataset will be used for evaluation. The project will be evaluated based on the rigor in

methodology and depth of understanding, in addition to the quality of the report and your performance in Viva.

Your project report (for Part 4) should be well structured and should include the following components.

1. An introduction to the problem setting,
2. The limitations of the basic VSM with appropriate examples from the dataset(s),
3. Your proposed approach(es) to address these issues,
4. A description of the dataset(s) used for experimentations,
5. The results obtained with a comparative study of your approach has improved the IR system, both qualitatively and quantitatively.

The latex template for the final report will be uploaded on Moodle. You are instructed to follow the template strictly.